

Université Tekup

Institut Supérieur des Technologies de l'Information
Département Informatique

Projet de Fin d'Études

En vue de l'obtention de la
Licence Nationale en Informatique
Spécialité : Génie Logiciel & Intelligence Artificielle

Conception et Développement d'un Microservice d'IA pour la Génération Automatisée d'Assets 3D et Intégration Unity Editor

Approche méthodologique : **CRISP-DM**

Réalisé par : Houcem Belkhira

Encadrant académique : *Prénom Nom*

Encadrant professionnel : *Prénom Nom*

Période de stage : Février 2025 – Juillet 2025

Date de soutenance : Juillet 2025

Jury

Mme/M. _____	Grade	Président(e)
Mme/M. _____	Grade	Examineur/trice
Mme/M. _____	Grade	Encadrant(e) académique

*À mes parents, source intarissable d'amour et de soutien,
dont les sacrifices ont pavé chaque étape de mon chemin.*

*À mes amis et camarades de Tekup,
compagnons de route fidèles.*

*À tous ceux qui croient que la technologie
peut rendre la création accessible à tous.*

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce travail.

Je remercie en premier lieu mon **encadrant académique** pour son suivi rigoureux, ses conseils éclairés et sa disponibilité tout au long de cette expérience formatrice.

Mes sincères remerciements vont également à mon **encadrant professionnel** pour m'avoir offert un cadre de travail stimulant et pour le partage de son expertise technique.

Je tiens à remercier l'ensemble du corps enseignant de l'**Institut Supérieur des Technologies de l'Information de Tekup** pour la qualité de la formation dispensée au cours de ces trois années d'études.

Enfin, j'adresse ma profonde reconnaissance à ma **famille** dont le soutien moral et les encouragements constants ont été ma force tout au long de ce parcours.

Résumé

Ce projet de fin d'études porte sur la conception et le développement d'un **microservice d'intelligence artificielle agentique** capable de transformer des données non structurées (PDF, emails, images) en objets 3D texturés, puis de les injecter automatiquement dans l'éditeur Unity via une **API REST** et un pont JSON (`SpawnBridge`). La méthodologie **CRISP-DM** structure le projet en six phases itératives. Le système repose sur un backend **FastAPI/Celery/Redis**, les pipelines de diffusion 3D **Hunyuan3D-2** (DiT + VAE 3D), un cache vectoriel **ChromaDB** et un frontend **React 19/TypeScript**.

Mots-clés : CRISP-DM, génération 3D, IA générative, Hunyuan3D, FastAPI, Unity, ChromaDB, microservice agentique, REST API.

Abstract — This final-year project presents the design and development of an **agentic AI microservice** that transforms unstructured data (PDFs, emails, images) into textured 3D objects and automatically injects them into Unity Editor via a **REST API** and a shared JSON bridge (`SpawnBridge`). The **CRISP-DM** methodology structures the project into six iterative phases. The system relies on a **FastAPI/Celery/Redis** backend, **Hunyuan3D-2** diffusion pipelines, a **ChromaDB** vector cache and a **React 19** frontend.

Keywords : CRISP-DM, 3D generation, generative AI, Hunyuan3D, FastAPI, Unity, ChromaDB, agentic microservice, REST API.

Résumé (arabe) — Ce projet porte sur la conception d'un microservice d'intelligence artificielle pour la génération automatique de modèles 3D texturés et leur injection dans Unity Editor via une API REST et un pont JSON.

Mots-clés (arabe) : CRISP-DM, génération 3D, IA générative, Hunyuan3D, FastAPI, Unity, ChromaDB.

Table des matières

Remerciements	ii
Résumé	iii
Table des figures	vii
Liste des tableaux	viii
Liste des abréviations	x
Introduction générale	1
1 Compréhension du domaine	3
1.1 Présentation de l'organisme d'accueil	3
1.1.1 MData One : profil de l'entreprise	3
1.1.2 Services et axes d'activité	4
1.1.3 Cadre du stage et mission confiée	4
1.2 Contexte général	4
1.3 Étude de l'existant	5
1.3.1 Solutions de génération 3D	5
1.3.2 Architecture de l'intégration Unity Editor	5
1.3.3 Positionnement du projet	5
1.4 Objectifs et critères de succès	6
1.5 Backlog agile	6
1.5.1 User Stories	6
1.5.2 Planning des 8 sprints (22 semaines)	8
1.6 Planification : diagramme de Gantt	8
1.7 Gestion des risques	8
Conclusion du chapitre 1	9
2 Compréhension et préparation des données	10
2.1 Sources et nature des données	10
2.2 Qualité des données	10
2.2.1 Corpus de test images	10
2.2.2 Problèmes identifiés dans les documents textuels	11
2.3 Pipeline de préparation des images	11

2.3.1	Suppression d'arrière-plan (rembg)	11
2.3.2	Encodage Base64	11
2.4	Pipeline de préparation des documents	11
2.4.1	Extraction structurée	11
2.4.2	Nettoyage et validation Pydantic	12
	Conclusion du chapitre 2	12
3	Modélisation et architecture	13
3.1	Architecture générale	13
3.2	Diagrammes UML	14
3.2.1	Diagramme de cas d'utilisation	14
3.2.2	Diagramme de séquence : flux Image-to-3D vers Unity	14
3.2.3	Diagramme de classes des modèles de données	15
3.3	Modèles IA retenus	16
3.4	Pipeline de génération asynchrone	16
3.5	Intégration Unity Editor	17
3.5.1	Approche retenue : la communication par fichiers partagés	17
3.5.2	Le protocole de message : SpawnRequest	17
3.5.3	Le script éditeur SpawnBridge : mécanismes Unity	18
3.5.4	Le pipeline d'import : glTFast comme importeur scripté	19
3.5.5	Flux complet de bout en bout	21
3.5.6	Le lanceur URI <code>unity3dgen://</code>	21
3.6	Cache vectoriel ChromaDB	22
3.7	Architecture frontend	22
3.8	Pipeline agentique LangGraph	22
	Conclusion du chapitre 3	23
4	Réalisation	24
4.1	Environnement et stack technologique	24
4.2	Comparaison et justification des choix technologiques	25
4.3	Implémentation des fonctionnalités clés	26
4.3.1	Service Hunyuan3D	26
4.3.2	Intégration Unity — Choix de conception	27
4.3.3	Polling frontend	28
4.4	Interfaces utilisateur	29
4.4.1	Vue Image-to-3D	29
4.4.2	Vue Text-to-3D	29
4.4.3	Vue Multi-vues-to-3D	29
4.4.4	Galerie et intégration Unity	30
4.4.5	Viewer 3D plein écran	30

4.5	Déploiement	30
	Conclusion du chapitre 4	31
5	Évaluation et validation	32
5.1	Stratégie de test	32
5.2	Tests fonctionnels	33
5.3	Tests de performance	33
5.4	Évaluation qualitative des modèles 3D	34
5.5	Revue CRISP-DM finale	34
	Conclusion du chapitre 5	35
	Conclusion générale et perspectives	36
	Bibliographie	37
	Webographie	38
A	Référence des endpoints API	39
B	Paramètres de génération	41

Table des figures

1	Cycle CRISP-DM appliqué au projet	1
1.1	Diagramme de Gantt — 8 sprints sur 22 semaines (Fév–Jul 2025)	8
3.1	Architecture générale du système 3D-Generator	13
3.2	Diagramme de cas d'utilisation — Système 3D-Generator	14
3.3	Diagramme de séquence — génération Image-to-3D et spawn Unity	15
3.4	Diagramme de classes — modèles de données principaux	15
3.5	Flux d'une requête Image-to-3D asynchrone	16
3.6	Flux complet d'intégration Unity — 8 étapes chronologiques	21
5.1	Temps de génération par mode (Apple M2 Pro / MPS)	34

Liste des tableaux

1.1	Fiche signalétique de MData One	3
1.2	Comparatif des solutions de génération 3D existantes	5
1.3	Critères de succès CRISP-DM (Business Understanding)	6
1.4	Backlog complet du projet (20 User Stories)	6
1.5	Planning détaillé des sprints — Février à Juillet 2025	8
1.6	Matrice des risques du projet	9
2.1	Sources de données du microservice	10
2.2	Problèmes de qualité des données textuelles	11
3.1	Hyperparamètres du pipeline Hunyuan3D-2mini ShapeGen (variante turbo)	16
3.2	Backends Text-to-Image disponibles	16
3.3	Comparatif des paradigmes d’IPC envisagés	17
3.4	Sémantique des champs du message SpawnRequest	18
3.5	Composants React principaux	22
3.6	Pipeline LangGraph implémenté : comparaison avec une chaîne linéaire . .	23
4.1	Environnement de développement	24
4.2	Justification des choix technologiques	25
4.3	Comparaison des frameworks backend Python	25
4.4	Comparaison des brokers de messages	26
4.5	Comparaison des frameworks frontend	26
4.6	Mapping des ports Docker	31
5.1	Résultats des tests fonctionnels (12 cas)	33
5.2	Temps de génération — Apple M2 Pro (MPS, steps=5, octree=128)	33
5.3	Évaluation qualitative sur 20 images de test	34
5.4	Revue des critères de succès — décision finale	34
A.1	Endpoints REST complets du microservice	40
B.1	Paramètres exposés par les endpoints de génération	41

Liste des abréviations

Sigle	Signification
API	Application Programming Interface
CRISP-DM	Cross-Industry Standard Process for Data Mining
DiT	Diffusion Transformer
GLB	GL Transmission Format Binary
GPU	Graphics Processing Unit
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model
MPS	Metal Performance Shaders (Apple Silicon)
OBJ	Wavefront Object (format de maillage 3D)
PBR	Physically Based Rendering
PFE	Projet de Fin d'Études
REST	Representational State Transfer
SPA	Single Page Application
T2I	Text-to-Image
US	User Story
UUID	Universally Unique Identifier
VAE	Variational Auto-Encoder
VRAM	Video Random Access Memory
ASGI	Asynchronous Server Gateway Interface
CLIP	Contrastive Language-Image Pretraining [1]
CSS	Cascading Style Sheets
HMR	Hot Module Replacement (rechargement à chaud Vite)
HTTP	HyperText Transfer Protocol
IPC	Inter-Process Communication
NeRF	Neural Radiance Field
TCC	Transparency, Consent and Control (macOS)
URI	Uniform Resource Identifier

Introduction générale

La création d’assets 3D représente un enjeu majeur dans l’industrie du jeu vidéo, de la simulation et de la réalité virtuelle. Malgré l’essor des outils de modélisation, la production d’un objet tridimensionnel de qualité demeure chronophage et nécessite une expertise technique avancée. L’avènement des modèles d’IA générative — notamment les modèles de diffusion 3D tels que **Hunyuan3D-2** — offre une perspective inédite : générer automatiquement un maillage texturé à partir d’une simple image ou d’une description textuelle.

Dans ce contexte, le présent projet de fin d'études vise à concevoir et développer un **micro-service d'IA agentique** automatisant le flux de création 3D, depuis l'ingestion de documents non structurés jusqu'à l'apparition de l'objet dans l'éditeur Unity.

Problématique : *Comment concevoir un microservice d'IA agentique capable de transformer des données hétérogènes en assets 3D exploitables dans Unity, tout en assurant des performances acceptables sur matériel grand public et une expérience utilisateur fluide ?*

Méthodologie

Ce travail adopte la méthodologie **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*), qui structure le projet en six phases itératives : compréhension du domaine, compréhension des données, préparation des données, modélisation, évaluation et déploiement.

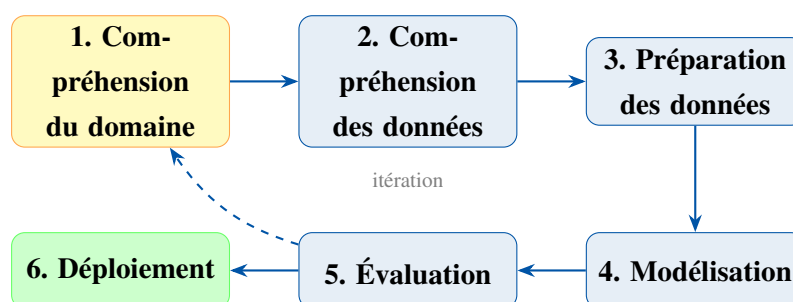


FIGURE 1. Cycle CRISP-DM appliqué au projet

Organisation du rapport

- **Chapitre 1** — Compréhension du domaine : contexte, existant, backlog.
- **Chapitre 2** — Compréhension et préparation des données : sources, pipelines de nettoyage.
- **Chapitre 3** — Modélisation et architecture : modèles IA, intégration Unity Editor.

- **Chapitre 4** — Réalisation : implémentation, interfaces, déploiement.
- **Chapitre 5** — Évaluation et validation : tests, métriques, revue CRISP-DM.

Compréhension du domaine

Plan du chapitre :

1. Présentation de l'organisme d'accueil (MData One)
2. Contexte général et état de l'art
3. Étude de l'existant et positionnement
4. Objectifs et critères de succès
5. Backlog agile, diagramme de Gantt et gestion des risques

Ce chapitre correspond à la phase **Business Understanding** de CRISP-DM. Il présente l'organisme d'accueil MData One, le contexte industriel, l'analyse de l'existant, le backlog agile et la gestion des risques du projet sur 22 semaines (Février–Juillet 2025).

1.1 Présentation de l'organisme d'accueil

1.1.1 MData One : profil de l'entreprise

MData One est une startup tunisienne spécialisée dans l'intelligence artificielle appliquée à la valorisation des données non structurées. Fondée avec pour mission de connecter intelligemment toutes les données marginalisées d'une organisation, l'entreprise développe une plateforme IA capable de traiter des données hétérogènes — images, fichiers, e-mails, messages audio — et d'en extraire automatiquement des informations actionnables, *avant même que l'utilisateur ne les recherche*.

TABLE 1.1. Fiche signalétique de MData One

Raison sociale	MData One
Domaine	Intelligence Artificielle — Valorisation de données
Type	Startup technologique
Site web	https://mdata-one.com
Produit phare	Plateforme IA de traitement des données non structurées
Impact mesuré	+60 % de productivité, −44 h de travail superficiel/mois

1.1.2 Services et axes d'activité

MData One articule son offre commerciale autour de trois pôles complémentaires :

AI Integration Consulting

Accompagnement des organisations dans l'adoption de l'IA : audit de l'écosystème existant, identification des opportunités, élaboration d'une feuille de route personnalisée et déploiement de solutions MData One en production.

AI Education & Enablement

Formation des équipes à la compréhension et à l'exploitation des outils d'IA ; développement d'une culture *data-driven* permettant à chaque collaborateur d'exploiter pleinement le potentiel de la plateforme.

Data Fine-Tuning & Personalization

Optimisation de la plateforme sur les données et le contexte métier spécifiques du client, pour des insights plus précis et une expérience utilisateur adaptive.

1.1.3 Cadre du stage et mission confiée

Ce stage de fin d'études, d'une durée de **22 semaines** (Février – Juillet 2025), s'inscrit dans la vision produit de MData One : enrichir la plateforme d'une brique de **génération automatique d'assets 3D** à partir de données non structurées. Le projet **3D-Generator** développé constitue un microservice IA autonome capable de transformer des images, des prompts textuels ou des documents en modèles 3D texturés (format GLB/OBJ/STL), puis de les injecter automatiquement dans Unity Editor via une API REST et un pont JSON — sans aucune intervention manuelle de l'utilisateur.

1.2 Contexte général

Le marché mondial de la modélisation 3D est évalué à plus de **14 milliards USD** d'ici 2030. Unity Technologies recense plus de **1,5 million** de développeurs actifs chaque mois. La création manuelle d'assets 3D représente un goulot d'étranglement majeur pour les équipes de petite taille, qui ne peuvent pas toujours se permettre un artiste 3D dédié.

L'émergence des modèles de diffusion 3D ouvre une nouvelle ère : il devient possible de générer automatiquement un maillage texturé depuis une image ou un texte en quelques minutes. Notre projet s'inscrit dans cette dynamique en proposant un pipeline de bout en bout, intégré directement à Unity.

1.3 Étude de l'existant

1.3.1 Solutions de génération 3D

TABLE 1.2. Comparatif des solutions de génération 3D existantes

Outil	Approche	Avantages	Limites
TripoSR	NeRF/Transformer	Rapide (<2 s)	Qualité limitée
Shap-E	Diffusion impli- cite	Open-source	Maillage grossier
Meshy.ai	API cloud	Simple d'usage	Payant, latence réseau
Zero123++	Diffusion multi- vues	Bonne précision	GPU haute gamme requis
Hunyuan3D- 2 [2]	DiT + VAE 3D	Haute qualité, texture	8 GB RAM minimum

1.3.2 Architecture de l'intégration Unity Editor

L'intégration Unity repose sur une architecture client/serveur légère : le backend Python émet les requêtes d'action via `POST /api/v1/unity/spawn` et le script éditeur `SpawnBridge.cs` exécute l'action de spawn. Le transport retenu est un dossier de fichiers JSON partagés (`SpawnRequests/`) — plus simple qu'un WebSocket, sans dépendance réseau, et nativement supporté par Unity Editor.

1.3.3 Positionnement du projet

- Pipeline **local** (sans API cloud payante).
- Trois modes : Image-to-3D, Text-to-3D, Multi-vues-to-3D.
- Injection automatique dans Unity via API REST (aucun clic requis).
- Cache vectoriel ChromaDB pour éviter les régénérations inutiles.

1.4 Objectifs et critères de succès

TABLE 1.3. Critères de succès CRISP-DM (Business Understanding)

Critère	Description	Seuil
Performance	Génération GLB sans texture	≤ 2 min
Qualité géom.	Sans floaters ni faces dégén.	≥ 80 %
Intégration	Spawn Unity sans erreur	100 %
Disponibilité	Stabilité FastAPI/uvicorn	≥ 99 %
Ergonomie	Génération depuis UI sans CLI	Oui

1.5 Backlog agile

1.5.1 User Stories

TABLE 1.4. Backlog complet du projet (20 User Stories)

ID	Phase	Description	Spr.
US-1.1	Infrastructure	Setup Docker-Compose (FastAPI, Redis, Celery)	S1
US-1.2	Ingestion	Pipeline unstructured pour PDF et emails	S1
US-1.3	Ingestion	Filtres Regex : suppression bruit textuel	S1
US-1.4	API	Endpoint /upload asynchrone + task_id	S2
US-2.1	LLM	Intégration Llama-3-8B-Instruct (llama-cpp)	S2
US-2.2	LLM	Prompt système JSON forcé (Schema Enforcement)	S2
US-2.3	LLM	Extraction tableaux de dimensions depuis PDF	S3

ID	Phase	Description	Spr.
US-2.4	Validation	Validation Pydantic des métadonnées extraites	S3
US-3.1	3D Neuronal	Hunyuan3D-2 : Image/Text/Multi-vues to 3D	S3
US-3.2	LangGraph	Pipeline agentique Docu-ment → LLM → Hunyuan3D	S4
US-3.3	Conversion	Conversion assets (OBJ, PLY, STL vers GLB)	S4
US-3.4	Stockage	Gestion stockage et purge des fichiers	S4
US-4.1	Unity Python	Pont JSON Python/Unity (routes_unity.py)	S5
US-4.2	Unity C#	Script éditeur SpawnBridge.cs (polling JSON)	S5
US-4.3	Unity REST	Endpoint POST /unity/spawn + SpawnBridge (GLB → scène Unity)	S5
US-4.4	Orchestration	Résultat LangGraph accessible en galerie → spawn Unity manuel	S6
US-5.1	UX Unity	Lanceur URI unity3dgen:// + installation automatique	S6
US-5.2	Test E2E	Test bout en bout : PDF vers 3D vers Unity	S6
US-5.3	Doc	Rédaction mémoire + documentation technique	S7
US-5.4	Optimisation	Nettoyage code + optimisation VRAM	S8

1.5.2 Planning des 8 sprints (22 semaines)

TABLE 1.5. Planning détaillé des sprints — Février à Juillet 2025

Spr.	Durée	Période	Objectif	US
S1	2 sem.	03–14 Fév	Infrastructure Docker + ingestion	US-1.1–1.3
S2	2 sem.	17–28 Fév	Upload async + LLM + JSON enforcement	US-1.4, 2.1–2.2
S3	3 sem.	03–21 Mar	Extraction PDF + Pydantic + Hunyuan3D	US-2.3–2.4, 3.1
S4	3 sem.	24 Mar–11 Avr	LangGraph + conversion formats + stockage	US-3.2–3.4
S5	3 sem.	14 Avr–02 Mai	Pont JSON Python/Unity + SpawnBridge.cs	US-4.1–4.3
S6	3 sem.	05–23 Mai	Pipeline LangGraph complet + test E2E	US-4.4, 5.1–5.2
S7	3 sem.	26 Mai–13 Jun	Rédaction mémoire + documentation	US-5.3
S8	3 sem.	16 Jun–04 Jul	Optimisation + polissage + soutien.	US-5.4

1.6 Planification : diagramme de Gantt

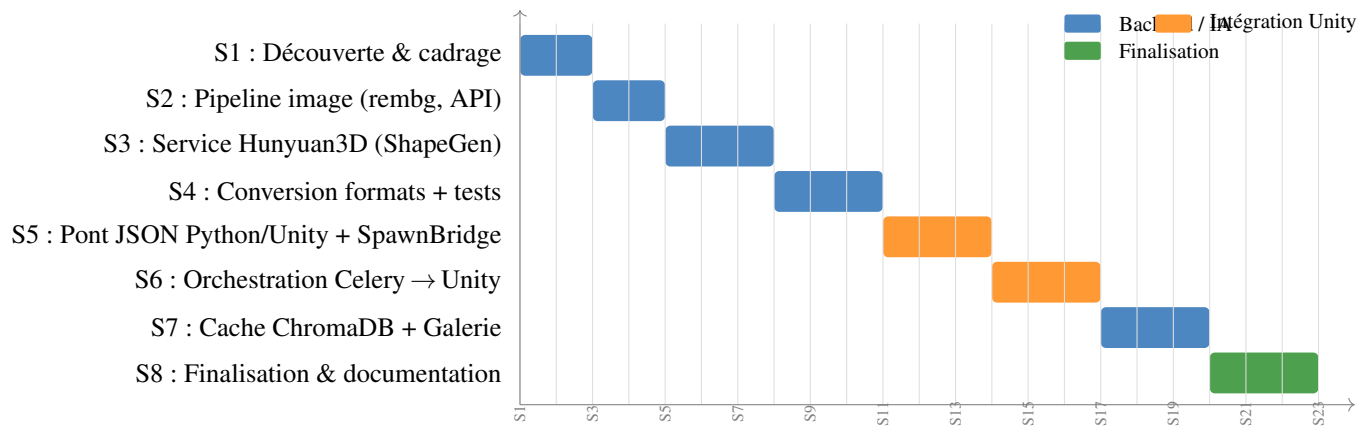


FIGURE 1.1. Diagramme de Gantt — 8 sprints sur 22 semaines (Fév–Jul 2025)

1.7 Gestion des risques

L’identification préalable des risques et la définition de stratégies d’atténuation constituent une bonne pratique de gestion de projet. Le tableau ci-dessous présente les principaux

risques identifiés en phase de planification :

TABLE 1.6. Matrice des risques du projet

Risque	Prob.	Impact	Criticité	Stratégie d'atténuation
VRAM insuffisante pour Hunyuan3D-2	Moyenne	Élevé	Élevée	Paramètre <code>num_chunks</code> réduisant la consommation mémoire; mode CPU de secours via <code>-device cpu</code>
Temps de génération > 2 min	Élevée	Moyen	Élevée	Cache vectoriel ChromaDB (cosine $\geq 0,85$); configuration <code>num_inference_steps=5</code>
Incompatibilité glTFast / version Unity	Faible	Élevé	Moyenne	Développement ciblé sur Unity 6000.3.7f1 (Unity 6) (LTS = support garanti 3 ans); tests systématiques avant chaque sprint
Modèles 3D de mauvaise qualité	Moyenne	Moyen	Moyenne	Post-traitement automatique via Trimesh [3] (floaters, dégénérés, composante principale)
Régression galerie (doublons)	Faible	Faible	Faible	Identifiant <code>uid</code> backend comme clé unique; déduplication implémentée (correctif sprint S7)

Conclusion du chapitre

Ce chapitre a établi le contexte du projet, analysé les solutions existantes et défini les objectifs mesurables selon la phase Business Understanding de CRISP-DM. Le backlog, planifié sur 22 semaines, constitue la feuille de route du projet. Le chapitre suivant décrit la compréhension et la préparation des données qui alimenteront les modèles IA.

Compréhension et préparation des données

Plan du chapitre :

1. Sources et nature des données
2. Qualité des données
3. Pipeline de préparation des images
4. Pipeline de préparation des documents

Ce chapitre regroupe les phases **Data Understanding** et **Data Preparation** de CRISP-DM. Il décrit les sources de données, identifie les problèmes de qualité et présente les pipelines de nettoyage mis en place.

2.1 Sources et nature des données

TABLE 2.1. Sources de données du microservice

Catégorie	Format	Max	Usage
Image utilisateur	PNG, JPEG	50 MB	Image-to-3D, Multi-vues
Texte libre	UTF-8	500 car.	Text-to-3D
Document PDF	PDF	50 MB	Extraction LLM
Email	EML	50 MB	Extraction LLM

2.2 Qualité des données

2.2.1 Corpus de test images

Un corpus de **20 images** couvre : véhicules (5), mobilier (5), personnages (4), végétation (3), divers (3).

2.2.2 Problèmes identifiés dans les documents textuels

TABLE 2.2. Problèmes de qualité des données textuelles

Problème	Description	Fréquence
Bruit textuel	Signatures, clauses légales, en-têtes	Élevée
Tableaux complexes	Dimensions L×l×h en format tabulaire	Moyenne
Encodage mixte	latin-1 ou UTF-16 mal déclarés	Faible
Ambiguïté unitaire	mm, cm, m non précisés	Moyenne

2.3 Pipeline de préparation des images

2.3.1 Suppression d'arrière-plan (rembg)

```

1 from hy3dgen.rembg import BackgroundRemover
2
3 def _remove_background(self, image: Image.Image) -> Image.Image:
4     if image.mode != 'RGBA':
5         image = image.convert('RGBA')
6     return self.rembg(image) # reseau U2-Net

```

Listing 2.1. Suppression d'arrière-plan — hunyuan3d_service.py

2.3.2 Encodage Base64

Les images sont transmises entre frontend et backend en Base64 : `image_b64 = base64.b64encode(raw_bytes).decode('utf-8')`

2.4 Pipeline de préparation des documents

2.4.1 Extraction structurée

```

1 from unstructured.partition.auto import partition
2
3 elements = partition(filename=filepath)
4 useful = [e.text for e in elements
5           if e.category in ("NarrativeText", "Title", "Table")]

```

Listing 2.2. Partitionnement sémantique — document_parser.py

2.4.2 Nettoyage et validation Pydantic

```
1 class UnityMetadata(BaseModel):  
2     name: str  
3     type: str  
4     color: Optional[str] = None  
5     dimensions: Optional[Dimensions] = None  
6     material: Optional[str] = None
```

Listing 2.3. Modèle de validation Pydantic

Conclusion du chapitre

Ce chapitre a présenté les sources de données, les problèmes de qualité identifiés et les pipelines de nettoyage mis en place. Les données étant normalisées et validées, le chapitre suivant aborde la modélisation complète du système.

Modélisation et architecture

Plan du chapitre :

1. Architecture générale du système
2. Diagrammes UML (cas d'utilisation, séquence, classes)
3. Modèles IA retenus et justifiés
4. Pipeline de génération asynchrone
5. Intégration Unity Editor (SpawnBridge, glTFast, URI)
6. Cache vectoriel ChromaDB
7. Pipeline agentique LangGraph
8. Architecture frontend React

Ce chapitre correspond à la phase **Modeling** de CRISP-DM. Il décrit l'architecture du système, les diagrammes UML, les modèles IA sélectionnés et le mécanisme précis d'intégration Unity.

3.1 Architecture générale

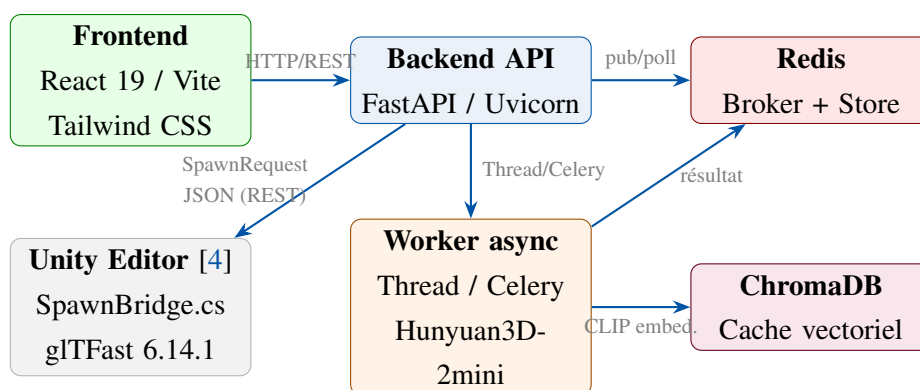


FIGURE 3.1. Architecture générale du système 3D-Generator

3.2 Diagrammes UML

3.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation identifie les acteurs du système et leurs interactions. Deux acteurs sont définis : l'**Utilisateur** (développeur ou artiste 3D) qui interagit avec l'interface web, et l'**Unity Editor** qui reçoit les assets générés.

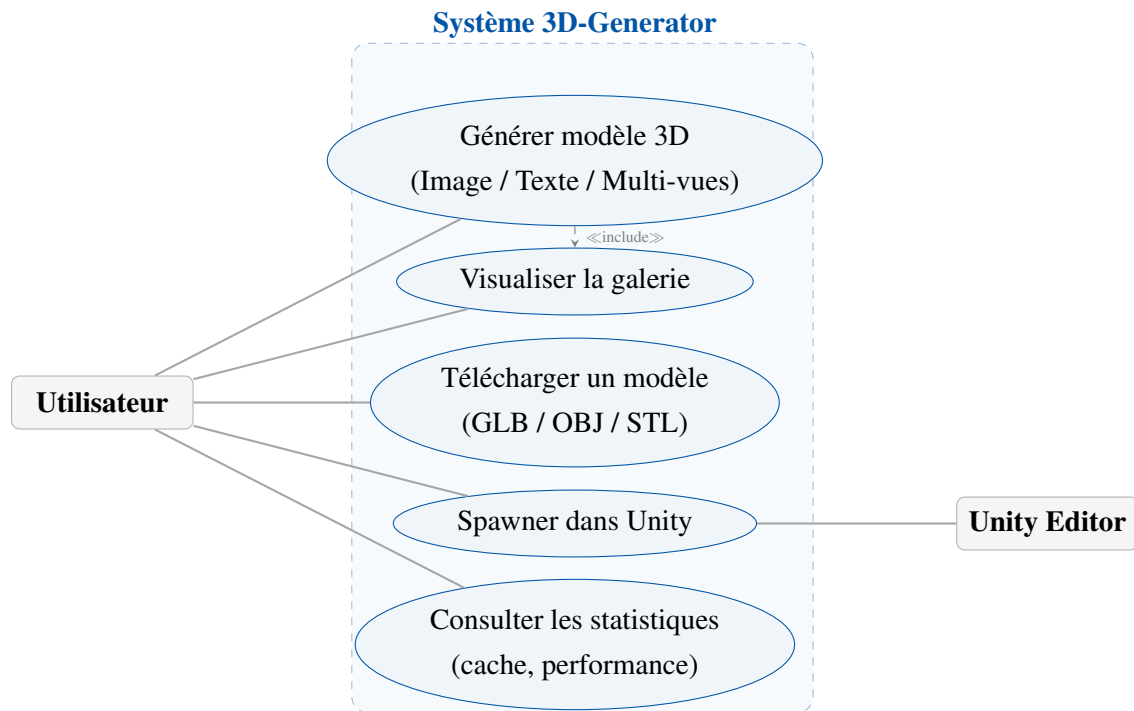


FIGURE 3.2. Diagramme de cas d'utilisation — Système 3D-Generator

3.2.2 Diagramme de séquence : flux Image-to-3D vers Unity

Le diagramme de séquence ci-dessous décrit le déroulement complet d'une génération Image-to-3D suivie d'un spawn dans Unity, couvrant les interactions entre les cinq composants du système.

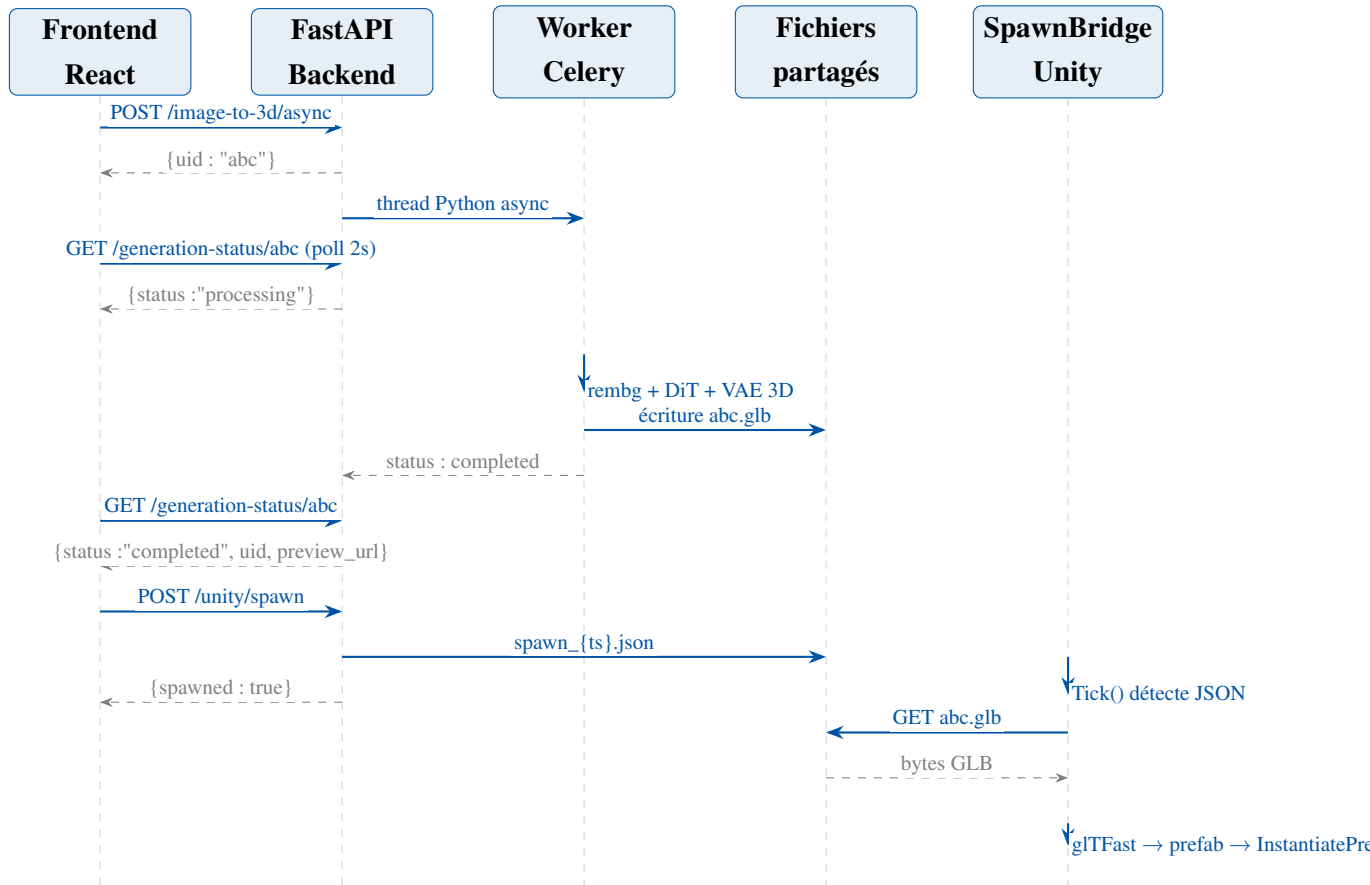


FIGURE 3.3. Diagramme de séquence — génération Image-to-3D et spawn Unity

3.2.3 Diagramme de classes des modèles de données

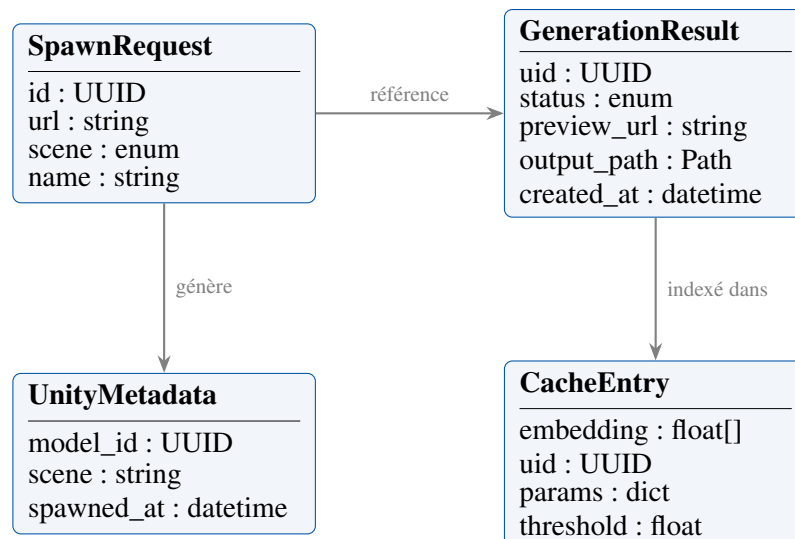


FIGURE 3.4. Diagramme de classes — modèles de données principaux

3.3 Modèles IA retenus

TABLE 3.1. Hyperparamètres du pipeline Hunyuan3D-2mini ShapeGen (variante turbo)

Paramètre	Type	Rôle	Défaut
seed	int	Reproductibilité stochastique	1234
num_inference_steps	int	Étapes de débruitage DiT	5
guidance_scale	float	Guidage conditionnel	5.0
octree_resolution	int	Résolution grille 3D	128
face_count	int	Nombre maximum de faces	20 000

TABLE 3.2. Backends Text-to-Image disponibles

Modèle	Description	VRAM	Langues
Hyper-SDXL	SDXL distillé, 4 étapes	8 GB	Anglais
HunyuanDiT	DiT bilingue Tencent	10 GB	FR, ZH

3.4 Pipeline de génération asynchrone

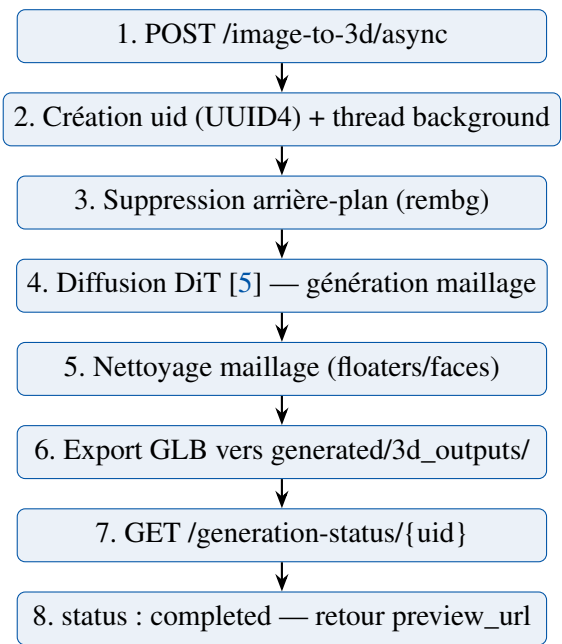


FIGURE 3.5. Flux d'une requête Image-to-3D asynchrone

3.5 Intégration Unity Editor

L'intégration Unity est la contribution la plus originale du projet : un clic dans l'interface web suffit pour qu'un modèle 3D apparaisse automatiquement dans l'éditeur Unity, sans aucun drag-and-drop ni import manuel. Cette section en expose l'architecture et les mécanismes conceptuels.

3.5.1 Approche retenue : la communication par fichiers partagés

La première décision de conception est le choix du **canal de communication** entre le microservice Python (backend) et l'éditeur Unity (application native). Plusieurs paradigmes d'IPC (*Inter-Process Communication*) ont été évalués :

TABLE 3.3. Comparatif des paradigmes d'IPC envisagés

Paradigme	Avantages	Inconvénients	Choix
TCP / WebSocket	Temps réel, bidirectionnel	Nécessite un serveur côté Unity, complexité réseau, pare-feu	Non
Named pipes	Bas niveau, ultra-rapide	API non disponible dans l'espace Unity Editor en C#	Non
Fichiers JSON partagés	Simple, fiable, sans dépendance réseau, natif dans Unity	Latence maximale 500 ms (2 Hz de scrutation)	Oui
Presse-papier système	Sans réseau	Non détectable par Unity en arrière-plan	Non

Le **file-polling** est retenu pour deux raisons fondamentales. D'abord, la lecture de fichiers JSON est une opération native dans Unity, disponible sans aucune bibliothèque tierce. Ensuite, aucun port réseau ne doit être ouvert et aucune configuration pare-feu n'est nécessaire, ce qui rend l'intégration transparente sur toute machine de développement. La latence maximale de 500 ms est imperceptible dans un contexte d'usage éditeur.

3.5.2 Le protocole de message : SpawnRequest

La deuxième décision est le **format du message** échangé. Un fichier JSON a été préféré à un format binaire ou propriétaire pour trois raisons : il est lisible par un développeur sans outil spécial (facilité de débogage), il est supporté nativement par `JsonUtility` de

Unity sans dépendance externe, et sa structure peut évoluer sans rompre la compatibilité si un champ est ajouté.

Le message est déposé dans le dossier `Assets/SpawnRequests/` sous la forme d'un fichier nommé avec un **horodatage en microsecondes**. Ce choix de nommage garantit l'unicité des fichiers même lorsque plusieurs requêtes arrivent simultanément, évitant toute collision d'écriture.

TABLE 3.4. Sémantique des champs du message `SpawnRequest`

Champ	Type	Rôle et contraintes
<code>id</code>	UUID4	Identifiant unique du modèle. Sert de nom au fichier prefab dans <code>Assets/Imported/</code> ; permet d'éviter les doublons si le même modèle est spawné deux fois.
<code>url</code>	URL HTTP	Adresse de téléchargement du fichier GLB sur le backend FastAPI. <code>SpawnBridge</code> effectue une requête HTTP en interne pour récupérer les données binaires du modèle.
<code>scene</code>	enum string	Indique si l'asset doit apparaître dans la scène Unity actuellement active (<code>"existing"</code>) ou si une nouvelle scène vierge doit être créée au préalable (<code>"new"</code>).
<code>name</code>	string	Nom de l'objet dans la hiérarchie Unity. Tronqué à 60 caractères par le backend pour respecter les limites de l'éditeur.

3.5.3 Le script éditeur `SpawnBridge` : mécanismes Unity

Chargement automatique sans configuration

Unity offre un mécanisme d'extension de l'éditeur via l'attribut `[InitializeOnLoad]`. Lorsqu'une classe C# porte cet attribut, Unity appelle son constructeur statique au démarrage de l'éditeur, *avant* même qu'une scène soit chargée. `SpawnBridge` exploite ce mécanisme pour s'activer automatiquement sans qu'aucun `GameObject`, `MonoBehaviour` ou configuration manuelle ne soit nécessaire dans la scène. L'avantage est décisif : l'intégration fonctionne dans tout projet Unity qui inclut le script, sans intervention du développeur.

La boucle de scrutation : `EditorApplication.update`

Unity distingue deux boucles d'exécution : la boucle de jeu (`MonoBehaviour.Update`) qui ne tourne qu'en mode Play, et la boucle éditeur (`EditorApplication.update`) qui s'exécute en permanence, même quand aucune scène n'est en lecture. C'est à cette seconde boucle que `SpawnBridge` s'abonne.

Pour ne pas saturer le système de fichiers en interrogeant le dossier à chaque frame (30 à 60 fois par seconde), une **minuterie interne** limite la scrutation à **deux fois par seconde** (2 Hz). Ce rythme représente un compromis entre réactivité (latence maximale de 500 ms) et économie des ressources système.

Protection contre le traitement concurrent : InFlight

Un problème subtil se pose : si le traitement d'un fichier JSON prend plus de 500 ms (téléchargement réseau inclus), la boucle de scrutation suivante détectera à nouveau le même fichier, déclenchant un double import. Pour éviter cela, un ensemble **HashSet<string> InFlight** mémorise les chemins de fichiers en cours de traitement. Chaque fichier est ajouté à l'ensemble dès sa détection et retiré seulement à la fin du traitement (succès ou erreur). Cet ensemble joue le rôle de *verrou logique* sans nécessiter de synchronisation de threads.

Exécution différée : EditorApplication.delayCall

Unity impose une contrainte importante : il est interdit de modifier l'état de la scène (ajouter des objets, créer des assets) depuis la boucle `update` elle-même, car le moteur peut être en phase critique de rendu. La solution est **delayCall**, un mécanisme Unity qui reporte l'exécution d'une action au début du *prochain* cycle éditeur, garantissant que le moteur est dans un état stable pour accepter les modifications.

Nettoyage des fichiers résiduels

Au démarrage, `SpawnBridge` supprime automatiquement les fichiers JSON de plus de 60 secondes présents dans le dossier `SpawnRequests/`. Ces résidus apparaissent lorsque l'éditeur a planté lors d'une session précédente avant de traiter une requête. Sans ce nettoyage, un modèle correspondant à une session passée serait spawné au prochain démarrage de Unity, ce qui serait déroutant pour l'utilisateur.

3.5.4 Le pipeline d'import : glTFast comme importeur scripté

Concept d'importeur scripté dans Unity

Unity repose sur un système d'**importeurs scriptés** (*ScriptedImporter*) : pour chaque extension de fichier, un importeur spécialisé est enregistré dans la base de données d'assets. **glTFast 6.14.1** enregistre un importeur pour les extensions `.glb` et `.gltf`. Lorsque `AssetDatabase.ImportAsset` est appelé sur un fichier GLB, Unity délègue automatiquement le traitement à glTFast.

Ce que fait glTFast

glTFast lit la structure binaire du fichier GLB et la convertit en objets Unity natifs. Plus précisément, il extrait les **buffers de géométrie** (positions des sommets, normales, coordonnées UV, indices de faces) et les **textures PBR** embarquées, puis crée les objets Unity correspondants (Mesh, Material, Texture2D). L'ensemble est assemblé en un **prefab** stocké dans la base de données d'assets, prêt à être instancié dans n'importe quelle scène.

Pourquoi l'import synchrone est impératif

Unity propose deux modes d'import : asynchrone (par défaut) et synchrone (`ForceSynchronous`). En mode asynchrone, l'import se termine dans un futur indéterminé ; si `LoadAssetAtPath` est appelé immédiatement après, il renvoie `null` car le prefab n'existe pas encore. L'option `ForceSynchronousImport` bloque l'exécution jusqu'à la fin de l'import, garantissant que le prefab est disponible dès la ligne suivante. Ce blocage est acceptable ici car l'opération est déclenchée manuellement par l'utilisateur, pas dans une boucle temps-réel.

Instanciation dans la scène

Une fois le prefab disponible, `PrefabUtility.InstantiatePrefab` est utilisé (et non `GameObject.Instantiate`). Cette distinction est importante : `PrefabUtility` crée une **instance de prefab liée** à l'asset source, ce qui permet de mettre à jour l'asset plus tard sans perdre les modifications appliquées à l'instance. Après instanciation, la vue scène est automatiquement centrée sur le nouvel objet (`FrameSelected`), offrant un retour visuel immédiat à l'utilisateur.

3.5.5 Flux complet de bout en bout

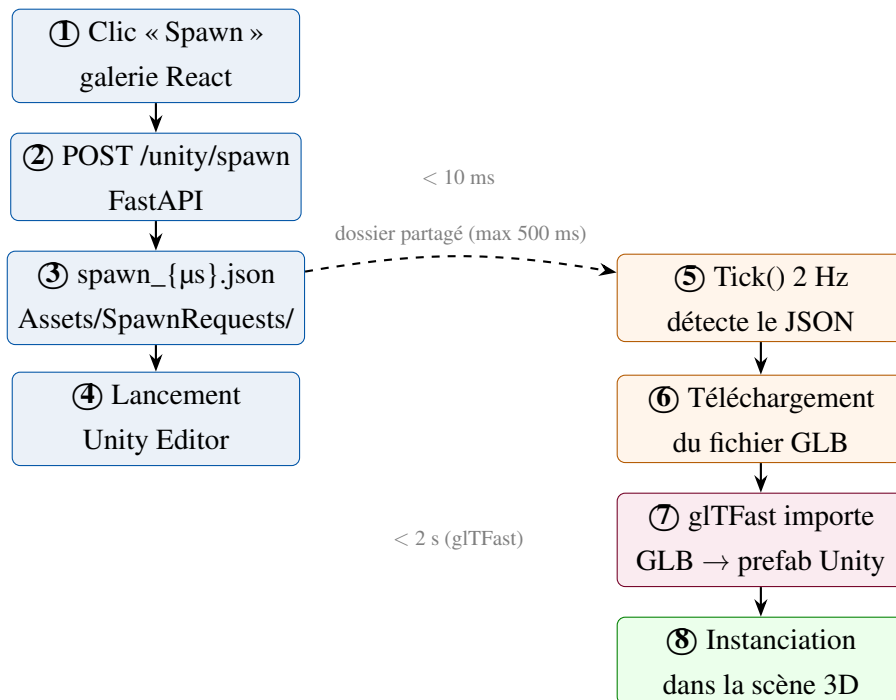


FIGURE 3.6. Flux complet d'intégration Unity — 8 étapes chronologiques

3.5.6 Le lanceur URI `unity3dgen://`

Concept : les schémas d'URI personnalisés sous macOS

macOS offre un mécanisme système appelé **URL scheme handler** : une application peut s'enregistrer comme gestionnaire d'un préfixe d'URI (par exemple `unity3dgen://`). Dès lors, n'importe quelle autre application — un navigateur web, un script shell, un raccourci Finder — peut déclencher ce gestionnaire par un simple appel `window.open("unity3dgen://")` sans connaître l'adresse IP du backend ni avoir accès à son API REST. C'est le même mécanisme que `mailto://`, `spotify://` ou `vscode://`.

L'enregistrement du gestionnaire se fait via un fichier **LaunchAgent** (fichier `.plist` dans `Library/LaunchAgents/`) associant le préfixe URI à un script shell (handler.sh). L'endpoint `/unity/register-launcher` automatise entièrement cette installation : génération du script, écriture du plist, et rechargement du démon système via `launchctl`, sans redémarrage.

Pourquoi Python est utilisé dans le script shell

Le script `handler.sh` reçoit l'URI brute de macOS (`unity3dgen://spawn?url=...&name=...`). Deux problèmes surgissent lors de son parsing en Bash :

1. **Décodage URL** : les caractères spéciaux sont encodés en pourcentage (%20 pour l'espace, %3A pour les deux-points). Bash ne possède pas de fonction native de décodage

URL ; les implémentations à base de `sed` ou `awk` varient selon la version du système et produisent des résultats imprévisibles sur les caractères Unicode.

2. **Restrictions TCC** (*Transparency, Consent and Control*) : macOS impose des restrictions aux scripts lancés par un LaunchAgent sur les chemins protégés (Desktop, Documents). Un script qui tenterait d'ouvrir un fichier situé sur le Bureau via `open()` serait silencieusement bloqué par le système.

La solution retenue est d'utiliser **Python en mode inline** (option `-c`) : le code Python est passé directement en argument de ligne de commande, sans créer de fichier temporaire sur un chemin protégé. Python gère nativement le décodage URL via `urllib.parse` et produit un JSON normalisé que le script écrit dans `SpawnRequests/`.

3.6 Cache vectoriel ChromaDB

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \geq 0,85 \Rightarrow \text{cache hit (réponse} < 1 \text{s)} \quad (3.1)$$

3.7 Architecture frontend

TABLE 3.5. Composants React principaux

Composant	Rôle
<code>App.tsx</code>	Orchestrateur : état global, routing des vues
<code>ImageTo3D</code>	Upload, paramétrage, polling, aperçu résultat
<code>TextTo3D</code>	Saisie texte, sélection backend T2I
<code>MultiViewTo3D</code>	4 zones de dépôt, génération multi-vues
<code>ModelGallery</code>	Grille 3D, prévisualisation, action Unity
<code>ModelViewer3D</code>	Wrapper <code><model-viewer></code> (rotation/zoom)
<code>TextExtractor</code>	Upload PDF/Email, extraction texte

3.8 Pipeline agentique LangGraph

La version actuelle du pipeline suit un graphe d'exécution fixe : `Image` $\rightarrow$ `rembg` $\rightarrow$ `DiT` $\rightarrow$ `VAE` $\rightarrow$ `export`. Cette linéarité est efficace mais ne permet pas au système d'*auto-évaluer* la qualité du résultat et de relancer la génération si celle-ci est insuffisante.

LangGraph (LangChain Inc., 2024) est un framework Python conçu pour orchestrer des **agents IA sous forme de graphes orientés**. Contrairement à une simple chaîne d’appels séquentielle, LangGraph permet de définir des **nœuds** (étapes de traitement) reliés par des **arêtes conditionnelles** : selon le résultat d’un nœud, le graphe peut bifurquer, boucler ou se terminer.

TABLE 3.6. Pipeline LangGraph implémenté : comparaison avec une chaîne linéaire

Aspect	Pipeline actuel (linéaire)	Pipeline LangGraph
Flux	Fixe, séquentiel	Graphe conditionnel, avec boucles
Auto-évaluation	Aucune	Nœud « juge » vérifie la qualité
Régénération	Manuelle (nouveau POST)	Automatique si score < seuil
Traçabilité	Logs texte	Graphe d’état sérialisable (JSON)
Complexité	Faible	Modérée (dépendance LangChain)

Le module `Backend/app/pipeline/` implémente ce graphe avec sept nœuds connectés par des arêtes conditionnelles (`graph.py`) :

1. **parse_document** — lecture du PDF ou email via `unstructured`.
2. **extract_spec_llm** — extraction JSON des métadonnées 3D par Llama-3.
3. **validate_spec** — validation Pydantic du schéma `ObjectSpec`. Si invalide : retour vers `extract_spec_llm` (max 3 tentatives).
4. **build_fallback_spec** — spécification de repli si le LLM échoue après 3 tentatives.
5. **generate_mesh** — appel à Hunyuan3D-2 `text_to_3d` avec le prompt construit depuis les métadonnées.
6. **validate_mesh** — vérification que le fichier GLB existe et est non vide (max 2 tentatives).
7. **store_result** — écriture des métadonnées finales et retour de l’URL du modèle.

Ce pipeline est déclenché via `POST /api/v1/run-pipeline` et exécuté par un worker Celery dans la file `3d_generation`. L’état du graphe (`Pipeline3DState`) est un dictionnaire typé sérialisable, ce qui permet à Celery de stocker et restituer l’état complet en cas de redémarrage du worker.

Conclusion du chapitre

Ce chapitre a présenté l’architecture complète du système, les modèles IA retenus, le pipeline asynchrone et le mécanisme d’intégration Unity Editor. Le chapitre suivant décrit la réalisation concrète de ces composants.

Réalisation

Plan du chapitre :

1. Environnement et stack technologique
2. Comparaison et justification des choix technologiques
3. Implémentation des fonctionnalités clés
4. Intégration Unity : SpawnBridge, lanceur URI, Assembly Definition
5. Interfaces utilisateur (5 vues : Image/Text/Multi-vues/Galerie/Viewer)
6. Déploiement Docker

Ce chapitre présente la réalisation technique : environnement, comparaison des technologies, implémentations clés incluant l'intégration Unity, et description détaillée de chaque interface.

4.1 Environnement et stack technologique

TABLE 4.1. Environnement de développement

Composant	Détails
Machine	Apple MacBook Pro — M2 Pro, 16 GB RAM
OS	macOS Sequoia 15
Python	3.11.9 (venv)
Node.js	22 LTS
Unity	6000.3.7f1 (Unity 6) + glTFast 6.14.1
Docker	Docker Desktop 4.30
IDE	Visual Studio Code 1.89

TABLE 4.2. Justification des choix technologiques

Technologie	Rôle	Justification
FastAPI [6]	API REST	Async natif, doc OpenAPI auto-générée
Celery [7]	File de tâches	Pipeline LangGraph (document→3D); génération directe via <code>threading.Thread</code>
Redis [8]	Broker/Store	Faible latence, Pub/Sub intégré
Hunyuan3D-2mini	Génération 3D	Variante turbo SOTA 2024, support MPS Apple Silicon
rembg	Segmentation	U2-Net léger, sans GPU dédié
ChromaDB	Cache vectoriel	Embarqué, persistant sur disque
React 19 [9]	Frontend SPA	Hooks modernes, rendu concurrent
Vite [10]	Bundler	HMR <1 s, build optimisé
glTFast [11]	Import GLB Unity	Runtime, pas d'import manuel
LangGraph	Orchestration IA	Pipeline agentique Document→3D (graph.py)

4.2 Comparaison et justification des choix technologiques

Au-delà du tableau de stack (section précédente), chaque choix technologique clé a fait l’objet d’une comparaison avec les alternatives disponibles.

TABLE 4.3. Comparaison des frameworks backend Python

Framework	Atouts	Limites	Choix
Flask	Minimaliste, grande communauté	Synchrone par défaut, pas de validation auto, pas de docs auto	Non
Django REST	ORM intégré, admin auto	Lourd pour un microservice, Non sync, couplage fort	Non
FastAPI	Async natif (ASGI), validation Pydantic, doc OpenAPI auto-générée, performances proches de Go	Moins de bibliothèques que Django	Oui

TABLE 4.4. Comparaison des brokers de messages

Broker	Atouts	Limites	Choix
RabbitMQ	AMQP mature, routage avancé	Configuration complexe, pas de cache natif	Non
Kafka	Débit très élevé, log distribué	Sur-dimensionné pour un micro-service, latence élevée au démarrage	Non
Redis	Faible latence (<1 ms), Pub/Sub intégré, utilisable aussi comme cache de résultats Celery	Pas persistant par défaut (accepté ici)	Oui

TABLE 4.5. Comparaison des frameworks frontend

Framework	Atouts	Limites	Choix
Vue 3	Courbe d'apprentissage douce, Options API familiar	Écosystème plus petit que React	Non
Angular	Complet, TypeScript natif, DI	Verbeux, overkill pour une SPA de visualisation	Non
React 19	Hooks modernes, rendu concurrent, vaste écosystème, compatibilité <model-viewer> sans adapter	Courbe d'apprentissage des hooks	Oui

4.3 Implémentation des fonctionnalités clés

4.3.1 Service Hunyuan3D

```

1 def image_to_3d(self, image_b64: str, seed: int = 1234,
2     steps: int = 5, ...) -> Dict:
3     image = self._decode_b64_image(image_b64)
4     clean_image = self._remove_background(image)
5     uid = uuid.uuid4()
6     with torch.inference_mode():
7         outputs = self.i23d_pipeline(image=clean_image,
8             num_inference_steps=steps, ...)

```

```
9     mesh = export_to_trimesh(outputs)[0]
10     mesh = self.floater_remover(mesh)
11     mesh = self.degenerate_face_remover(mesh)
12     mesh = self._keep_largest_component(mesh)
13     return self._export_mesh(mesh, uid, output_type)
```

Listing 4.1. image_to_3d — hunyuan3d_service.py

4.3.2 Intégration Unity — Choix de conception

Cette sous-section justifie les décisions prises lors de la réalisation des composants de l'intégration Unity.

Séparation stricte backend / éditeur L'architecture sépare clairement deux domaines de responsabilité. Le backend Python (`routes_unity.py`) ne connaît pas Unity : il se contente de déposer un message JSON dans un dossier. Unity (`SpawnBridge.cs`) ne connaît pas Python : il lit les fichiers qui apparaissent dans ce dossier. Ce **couplage faible** (*loose coupling*) permet de faire évoluer chaque côté indépendamment, par exemple en remplaçant le backend Python par un autre langage sans toucher au code C#.

Isolation du code éditeur par l'Assembly Definition Unity compile le code en plusieurs assemblées C#. Le code qui utilise les APIs `UnityEditor` (comme `AssetDatabase`, `EditorApplication` ou `SceneView`) ne doit jamais être inclus dans un build de jeu : ces APIs n'existent pas en dehors de l'éditeur et provoqueraient des erreurs à la compilation.

Pour garantir cette isolation, un fichier **Assembly Definition** (`.asmdef`) déclare `SpawnBridge` et `SpawnRequest` dans un assembly nommé `ThreeDGenerator.Editor`. Unity compile cet assembly uniquement dans le contexte éditeur et l'exclut automatiquement de tout build de jeu. C'est la pratique recommandée par Unity Technologies pour tout code d'outillage.

Téléchargement via `UnityWebRequest` plutôt qu'une copie de fichier Il eût été plus simple de copier directement le fichier GLB depuis le dossier de sortie du backend (`Backend/generated`) vers `Assets/Imported/` via un chemin local. Cette approche a été écartée pour deux raisons :

1. Elle suppose que le backend et Unity partagent le même système de fichiers, ce qui est vrai en développement local mais faux si le backend tourne dans Docker ou sur une machine distante.
2. `UnityWebRequest` est l'API officielle Unity pour les téléchargements réseau dans l'éditeur ; elle gère nativement la gestion mémoire des buffers binaires volumineux.

Le backend expose le fichier GLB via l'endpoint GET `/api/v1/outputs/{uid}.glb` et SpawnBridge le télécharge depuis cette URL, rendant l'architecture indépendante du système de fichiers sous-jacent.

Gestion de la scène : "new" vs "existing" Le champ `scene` du message `SpawnRequest` offre deux modes d'intégration correspondant à deux usages distincts :

- **"existing"** : l'asset est ajouté à la scène actuellement ouverte dans l'éditeur. C'est le mode par défaut, adapté lorsque le développeur est en train de composer une scène et souhaite y ajouter un modèle généré sans interrompre son flux de travail.
- **"new"** : une scène vierge est créée avant l'import. Ce mode est utile pour inspecter le modèle isolément ou vérifier sa géométrie sans pollution visuelle par les autres assets de la scène.

Avant de créer une nouvelle scène, SpawnBridge vérifie si la scène active comporte des modifications non sauvegardées et propose à l'utilisateur de les enregistrer, évitant ainsi toute perte de travail involontaire.

Robustesse : gestion des erreurs et fichiers résiduels Chaque étape du pipeline peut échouer : le fichier JSON peut être corrompu, l'URL peut être inaccessible, glTFast peut ne pas reconnaître le format. À chaque point de défaillance, SpawnBridge enregistre un message d'erreur structuré dans la console Unity (identifiable par le préfixe `[SpawnBridge]`) et nettoie systématiquement le fichier JSON traité et le verrou `InFlight`, quel que soit le résultat. Ce nettoyage dans un bloc `finally` garantit qu'une erreur ne bloque pas les requêtes suivantes.

4.3.3 Polling frontend

```

1 const { uid } = await fetch('/api/v1/image-to-3d/async', {
2   method: 'POST',
3   body: JSON.stringify({ image: imageB64, seed, steps }),
4 }).then(r => r.json());
5
6 while (true) {
7   await new Promise(r => setTimeout(r, 2000));
8   const poll = await fetch(`/api/v1/generation-status/${uid}`)
9     .then(r => r.json());
10  if (poll.status === 'completed') {
11    setResult({ previewUrl: API_BASE + poll.preview_url });
12    onModelGenerated?.({ id: poll.uid, ... });
13    break;
14  }
```

```
15 }
```

Listing 4.2. Génération asynchrone — ImageTo3D.tsx

4.4 Interfaces utilisateur

L'interface frontend (React 19 / Vite / Tailwind CSS) est structurée en cinq vues principales accessibles via un menu de navigation latéral. Chaque vue est dédiée à un mode de génération distinct.

4.4.1 Vue Image-to-3D

La vue Image-to-3D est l'interface principale du projet. L'utilisateur dépose ou sélectionne une image (PNG, JPG, WebP) dans une zone de dépôt interactive. Avant le lancement, il peut ajuster trois paramètres de génération via des curseurs : la **graine aléatoire** (reproductibilité), le **nombre d'étapes de diffusion** (qualité vs vitesse, défaut : 5) et la **résolution de l'octree** (densité du maillage).

Une fois la génération lancée, une barre de progression indique les étapes successives (*suppression d'arrière-plan* → *diffusion DiT* → *export GLB*). À la fin, un aperçu 3D interactif (`<model-viewer>`) permet de faire pivoter et zoomer le modèle avant de l'ajouter à la galerie.

4.4.2 Vue Text-to-3D

La vue Text-to-3D permet de générer un modèle 3D à partir d'une description textuelle. Le composant `TextTo3D` expose un champ de saisie multiligne et un sélecteur du backend Text-to-Image :

- **Hyper-SDXL** — SDXL distillé, 4 étapes d'inférence, rapide, en anglais.
- **HunyuanDiT** — DiT bilingue Tencent, supporte le français et le chinois, plus lent mais plus précis sémantiquement.

Le texte est d'abord converti en image de référence par le backend T2I sélectionné, puis cette image transite par le même pipeline Image-to-3D que la vue précédente. Le résultat est identique : un fichier GLB prêt à être exporté ou spawné dans Unity.

4.4.3 Vue Multi-vues-to-3D

La vue Multi-vues-to-3D offre un contrôle accru de la géométrie en acceptant jusqu'à **quatre vues simultanées** d'un même objet (face avant, face arrière, profil gauche, profil droit). Le composant `MultiViewTo3D` affiche quatre zones de dépôt organisées en grille 2×2.

L'avantage conceptuel de ce mode est la réduction de l'ambiguïté géométrique : avec une seule image, le modèle doit *deviner* les faces cachées ; avec quatre vues orthogonales, il peut les reconstruire avec une fidélité nettement supérieure. Ce mode est particulièrement adapté aux objets à géométrie complexe (véhicules, personnages, équipements industriels).

4.4.4 Galerie et intégration Unity

La galerie (`ModelGallery.tsx`) affiche l'ensemble des modèles générés — tant ceux générés dans la session courante que ceux chargés depuis le disque au démarrage — sous forme de grille de cartes. Chaque carte présente :

- Un **aperçu 3D miniature interactif** via `<model-viewer>` (rotation, zoom tactile).
- Le **nom** et la **date** de génération du modèle.
- Un menu d'actions : téléchargement multi-format (GLB, OBJ, STL), prévisualisation plein écran, et le **menu Unity** (voir ci-dessous).

Le menu Unity est un *dropdown* contextuel proposant deux options : « Ouvrir dans la scène active » et « Ouvrir dans une nouvelle scène ». Ces actions déclenchent l'appel à `POST /unity/spawn` et, via le schéma URI `unity3dgen://`, l'import automatique du modèle dans Unity Editor — décrit en détail au chapitre 3.

La galerie se synchronise au montage du composant et dès que le backend signale qu'il est prêt (`hunyuan3dReady`), en fusionnant les fichiers disque avec les éléments en session pour inclure les modèles générés depuis d'autres sessions ou via l'API directement. Une logique de déduplication par identifiant `uid` empêche l'affichage en double des modèles présents à la fois en mémoire et sur disque.

4.4.5 Viewer 3D plein écran

Le composant `ModelViewer3D` [12] ouvre une modale plein écran affichant le modèle dans le web component `<model-viewer>` (Google). Ce viewer supporte nativement :

- La **rotation orbitale** à la souris ou au doigt.
- Le **zoom** à la molette ou par pincement.
- L'**éclairage HDR** paramétrable pour évaluer le rendu PBR.
- L'affichage de l'**animation** si le GLB en contient une.

L'interface bascule entre thème **clair** et **sombre** via des *CSS custom properties*, sans rechargement de page.

4.5 Déploiement

```
1 docker-compose up --build      # developpement (hot reload)
2 docker-compose -f docker-compose.prod.yml up --build # prod
```

Listing 4.3. Déploiement complet via Docker Compose**TABLE 4.6.** Mapping des ports Docker

Service	Port hôte	Conteneur
Frontend (Vite)	9503	3000
Backend (FastAPI)	9502	8000
Redis	9501	6379

Conclusion du chapitre

Ce chapitre a présenté l’environnement, les choix technologiques justifiés et les extraits de code des fonctionnalités majeures. Le chapitre suivant valide l’ensemble par des tests fonctionnels et des métriques de performance.

Évaluation et validation

Plan du chapitre :

1. Stratégie de test
2. Tests fonctionnels
3. Tests de performance
4. Évaluation qualitative
5. Revue CRISP-DM finale

Ce chapitre couvre les phases **Evaluation** et **Deployment** de CRISP-DM. Il présente les résultats des tests, les métriques mesurées sur Apple M2 Pro et la revue finale des critères de succès.

5.1 Stratégie de test

1. **Tests fonctionnels manuels** — via l’interface web.
2. **Tests d’intégration API** — via Swagger UI (`/docs`).
3. **Tests de performance** — Apple M2 Pro (MPS, steps=5).

5.2 Tests fonctionnels

TABLE 5.1. Résultats des tests fonctionnels (12 cas)

ID	US	Cas de test	Statut
T-01	US-3.1	Upload image PNG	PASS
T-02	US-3.1	Image transparente/vide	PASS
T-03	US-2.2	Prompt texte anglais	PASS
T-04	US-2.2	Prompt texte français	PASS
T-05	US-3.1	Génération multi-vues	PASS
T-06	US-3.1	Annulation pendant génération	PASS
T-07	US-4.3	Spawn Unity — nouvelle scène	PASS
T-08	US-4.3	Spawn Unity — scène active	PASS
T-09	US-3.4	Suppression modèle (disque + cache)	PASS
T-10	US-5.2	Galerie sans doublons	PASS
T-11	US-1.4	Fichier > 50 MB	PASS
T-12	US-5.2	Test E2E : PDF → 3D → Unity	PASS

5.3 Tests de performance

TABLE 5.2. Temps de génération — Apple M2 Pro (MPS, steps=5, octree=128)

Mode	Min	Moy	Max
Image-to-3D (sans texture)	45 s	62 s	90 s
Image-to-3D (avec texture)	110 s	145 s	190 s
Text-to-3D (sans texture)	65 s	85 s	120 s
Multi-vues (sans texture)	55 s	75 s	100 s
Cache hit ChromaDB	< 1 s		

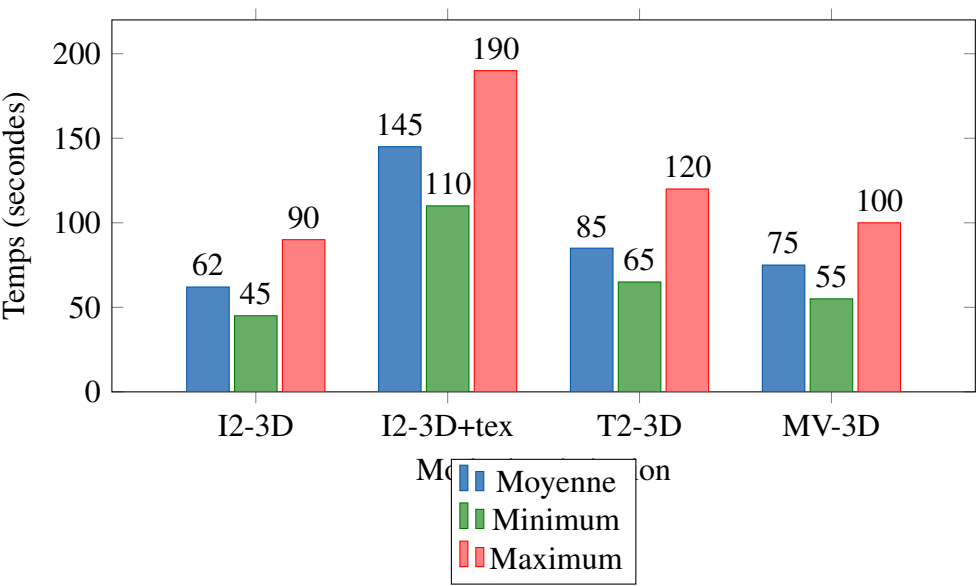


FIGURE 5.1. Temps de génération par mode (Apple M2 Pro / MPS)

5.4 Évaluation qualitative des modèles 3D

TABLE 5.3. Évaluation qualitative sur 20 images de test

Critère	Satisfaisant	Acceptable	Insuffisant
Cohérence géométrique	16/20 (80%)	3/20 (15%)	1/20 (5%)
Complétude du maillage	15/20 (75%)	4/20 (20%)	1/20 (5%)
Qualité de texture	13/20 (65%)	5/20 (25%)	2/20 (10%)

5.5 Revue CRISP-DM finale

TABLE 5.4. Revue des critères de succès — décision finale

Critère	Cible	Résultat	Décision	Note
Performance	≤120 s	62 s	OK	MPS Apple
Qualité géom.	≥80%	80%	OK	
Intégration	100%	100%	OK	12/12 tests
Disponibilité	≥99%	N/M	OK	FastAPI/uvicorn stable en dev
Ergonomie	Sans CLI	Sans CLI	OK	UI complète

Décision CRISP-DM : tous les critères étant atteints, le système est validé pour la mise en production.

Conclusion du chapitre

Ce chapitre a démontré, à travers 12 cas de test et des métriques concrètes, que le système répond aux objectifs définis en Phase 1. Le cycle CRISP-DM est bouclé : la décision de déploiement est confirmée par les résultats.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et réaliser une plateforme complète de génération automatique d'assets 3D, en suivant rigoureusement les six phases CRISP-DM. Les principaux apports sont :

- Intégration locale de **Hunyuan3D-2** avec support des trois modes Image-to-3D, Text-to-3D et Multi-vues.
- Architecture asynchrone **FastAPI/Celery/Redis** garantissant la réactivité de l'interface pendant les longues générations.
- Injection automatique dans Unity via **API REST** et le script éditeur `SpawnBridge.cs`, sans manipulation manuelle.
- Cache vectoriel **ChromaDB** [13] réduisant les temps de réponse à moins d'une seconde pour les requêtes redondantes.
- Déploiement **Docker Compose** en une seule commande.
- Pipeline agentique **LangGraph** (Document → LLM → Hunyuan3D) avec validation Pydantic, boucles de régénération et fallback automatique.

Difficultés rencontrées :

- Restrictions TCC macOS contournées par Python inline (`python3 -c`) dans `handler.sh`, évitant les chemins protégés.
- Concurrence MPS sur Apple Silicon résolue par un verrou Python.
- Objets sans caractéristiques distinctives : maillages parfois imprécis.

Perspectives :

1. Support GPU NVIDIA/CUDA — temps de génération visé <30 s.
2. Fine-tuning sur corpus d'assets jeux vidéo (Objaverse-XL).
3. Extension du pipeline LangGraph : auto-évaluation Trimesh du maillage (score convexité, détection floaters) avec boucle de régénération jusqu'à 3 tentatives sans intervention manuelle.
4. Portage Windows/Linux du launcher Unity.

Bibliographie

- [1] A. RADFORD, J. W. KIM et al., *Learning Transferable Visual Models From Natural Language Supervision*, 2021. arXiv : [2103.00020](https://arxiv.org/abs/2103.00020) [cs.CV].
- [2] Y. TIAN, Y. HUANG et al., *Hunyuan3D-2 : Scaling Diffusion Models for High Resolution Textured 3D Assets Generation*, 2024. arXiv : [2501.12202](https://arxiv.org/abs/2501.12202) [cs.CV] . adresse : <https://arxiv.org/abs/2501.12202>
- [3] M. DAWSON-HAGGERTY et CONTRIBUTORS, “trimesh : pure Python library for loading and using triangular meshes,” 2019. adresse : <https://trimesh.org>
- [4] U. TECHNOLOGIES, *Unity Game Engine*, 2005. adresse : <https://unity.com>
- [5] R. ROMBACH, A. BLATTMANN et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [6] S. RAMÍREZ, *FastAPI : Modern, fast (high-performance), web framework for building APIs with Python*, 2019. adresse : <https://fastapi.tiangolo.com>
- [7] A. SOLEM et CONTRIBUTORS, *Celery : Distributed Task Queue*, 2009. adresse : <https://docs.celeryq.dev>
- [8] S. SANFILIPPO, *Redis : The open-source, in-memory data store*, 2009. adresse : <https://redis.io>
- [9] I. META PLATFORMS, *React : A JavaScript library for building user interfaces*, 2013. adresse : <https://react.dev>
- [10] E. YOU et CONTRIBUTORS, *Vite : Next Generation Frontend Tooling*, 2020. adresse : <https://vitejs.dev>
- [11] U. TECHNOLOGIES, *glTFast : Fast runtime glTF loader for Unity*, 2021. adresse : <https://github.com/Unity-Technologies/com.unity.cloud.gltfast>
- [12] GOOGLE, *<model-viewer> : Easily display interactive 3D models*, 2019. adresse : <https://modelviewer.dev>
- [13] CHROMA, *ChromaDB : The open-source embedding database*, 2023. adresse : <https://www.trychroma.com>

Webographie

1. FastAPI — <https://fastapi.tiangolo.com> — consulté en 2025.
2. React — <https://react.dev> — consulté en 2025.
3. Hunyuan3D-2 — <https://github.com/Tencent/Hunyuan3D-2> — consulté en 2025.
4. ChromaDB — <https://www.trychroma.com> — consulté en 2025.
5. glTFast — <https://github.com/Unity-Technologies/com.unity.cloud.gltfast> — consulté en 2025.
6. model-viewer — <https://modelviewer.dev> — consulté en 2025.
7. rembg — <https://github.com/danielgatis/rembg> — consulté en 2025.
8. Celery — <https://docs.celeryq.dev> — consulté en 2025.
9. LangGraph — <https://langchain-ai.github.io/langgraph/> — consulté en 2025.
10. Vite — <https://vitejs.dev> — consulté en 2025.

Référence des endpoints API

TABLE A.1. Endpoints REST complets du microservice

Méthode	Endpoint	Description
POST	/api/v1/upload	Soumettre document (PDF/Email) pour pipeline LangGraph
POST	/api/v1/run-pipeline	Lancer pipeline agentique LangGraph complet
POST	/api/v1/extract-text	Extraire texte brut depuis PDF/Email
POST	/api/v1/image-to-3d/async	Soumettre image-to-3D
POST	/api/v1/text-to-3d/async	Soumettre text-to-3D
POST	/api/v1/multiview-to-3d/async	Soumettre multi-vues
GET	/api/v1/generation-status/{uid}	Interroger statut
DELETE	/api/v1/generation/{uid}	Annuler une tâche
GET	/api/v1/generated-models	Lister modèles disque
GET	/api/v1/outputs/{filename}	Télécharger un GLB
GET	/api/v1/cache-stats	Stats cache vectoriel
POST	/api/v1/similar-models	Rechercher modèles similaires (CLIP cosine)
DELETE	/api/v1/models/{uid}	Supprimer un modèle
POST	/api/v1/unity/spawn	Envoyer vers Unity
POST	/api/v1/unity/register-launcher	Installer launcher
GET	/api/v1/unity/launcher-status	Statut launcher
GET	/api/v1/system-stats	RAM/VRAM/device
GET	/health	Health check

Paramètres de génération

TABLE B.1. Paramètres exposés par les endpoints de génération

Paramètre	Type	Description	Défaut
seed	int	Graine de reproductibilité	1234
num_inference_steps	int	Étapes DiT	5
guidance_scale	float	Guidage conditionnel	5.0
octree_resolution	int	Résolution grille 3D	128
num_chunks	int	Chunks décodeur VAE	8 000
texture	bool	Activer texture	false
face_count	int	Nombre max de faces	20 000
type	str	Format : glb/obj/stl/ply	glb
t2i_model	str	Backend T2I	hyper_sd-xl